Hybrid Search & Retrieval Performance

A technical whitepaper on multi-vector retrieval, RRF reranking, full-text search, and measured performance

本白皮书为资料汇编：所有正文段落逐字取自 Milvus 官方文档（v2.6.x）与 Zilliz 官方基准页面，所有数值均为官方公布的实测结果，出处逐节标注。编排者为语料整理脚本，未改写任何事实。

Compiled: October 2026 · Sources listed in §6 References

Table of Contents

1 Why hybrid search
2 Reranking strategies in Milvus
3 RRF Ranker: mechanism and parameters
4 Sparse vectors, full-text search and the BM25 metric
5 Measured performance
5.1 VectorDBBench leaderboard (Cohere Medium 1M, 768-d)
5.2 Milvus 2.2 scale-up / scale-out benchmark
6 References

1 Why hybrid search

In the realm of information retrieval and generative AI, a reranker is an essential tool that optimizes the order of results from initial searches. Rerankers differ from traditional embedding models by taking a query and document as input and directly returning a similarity score instead of embeddings. This score indicates the relevance between the input query and document.

Rerankers are often employed after the first stage retrieval, typically done via vector Approximate Nearest Neighbor (ANN) techniques. While ANN searches are efficient at fetching a broad set of potentially relevant results, they might not always prioritize results in terms of actual semantic closeness to the query. Here, rerankers is used to optimize the results order using deeper contextual analyses, often leveraging advanced machine learning models like BERT or other Transformer-based models. By doing this, rerankers can dramatically enhance the accuracy and relevance of the final results presented to the user.

Source: milvus.io docs v2.6.x — Rerankers Overview (site/en/rerankers/rerankers-overview.md)

2 Reranking strategies in Milvus

Milvus supports hybrid search over multiple vector fields and merges the candidate lists with reranking strategies. The official documentation describes two families of strategies: reranking based on ranking positions (Reciprocal Rank Fusion, RRF) and reranking based on re-scored similarity (WeightedRanker), plus model-based rerankers (BGE, Cross Encoder, Voyage, Cohere, Jina AI) integrated through the PyMilvus model library.

The table below is reproduced verbatim from the official Rerankers Overview page:

Rerank Function	API or Open-sourced
BGE	Open-sourced
Cross Encoder	Open-sourced
Voyage	API
Cohere	API
Jina AI	API

Source: milvus.io docs v2.6.x — Rerankers Overview

3 RRF Ranker: mechanism and parameters

Reciprocal Rank Fusion (RRF) Ranker is a reranking strategy for Milvus hybrid search that balances results from multiple vector search paths based on their ranking positions rather than their raw similarity scores. Like a sports tournament that considers players' rankings rather than individual statistics, RRF Ranker combines search results based on how highly each item ranks in different search paths, creating a fair and balanced final ranking.

The main workflow of the RRFRanker strategy is: (1) Collect Search Rankings — collect the rankings of results from each path of vector search (rank_1, rank_2); (2) Merge Rankings — convert the rankings according to a formula; (3) Aggregate Rankings — re-rank the search results based on the combined rankings to produce the final results.

The calculation formula involves N, which represents the number of retrievals. $rank_i(d)$ is the ranking position of document d generated by the i-th retriever. k is a smoothing parameter

typically set at 60:

$$score_{RRF}(d) = \Sigma_{i=1..N} 1 / (k + rank_i(d))$$

When using the RRF reranking strategy, you need to configure the parameter k. It is a smoothing parameter that can effectively alter the relative weights of full-text search versus vector search. The default value of this parameter is 60, and it can be adjusted within a range of (0, 16384). The recommended value is between [10, 100]. While k=60 is a common choice, the optimal k value can vary depending on your specific applications and datasets.

Source: milvus.io docs v2.6.x — RRF Ranker (site/en/userGuide/embeddings-reranking/reranking/rrf-ranker.md)

4 Sparse vectors, full-text search and the BM25 metric

Currently, Milvus supports these types of similarity metrics: Euclidean distance (L2), Inner Product (IP), Cosine Similarity (COSINE), JACCARD, HAMMING, and BM25 (specifically designed for full text search on sparse vectors).

For vector fields of the SPARSE_FLOAT_VECTOR type, use the BM25 metric type only when performing full text search. The following mapping is reproduced verbatim from the official Metric Types page:

Field Type	Dimension Range	Supported Metric Types	Default
FLOAT_VECTOR	2-32,768	COSINE, L2, IP	COSINE
FLOAT16_VECTOR	2-32,768	COSINE, L2, IP	COSINE
BFLOAT16_VECTOR	2-32,768	COSINE, L2, IP	COSINE
INT8_VECTOR	2-32,768	COSINE, L2, IP	COSINE
SPARSE_FLOAT_VECTOR	No need to specify the dimension.	IP, BM25 (used only for full text search)	IP
BINARY_VECTOR	8-32,768*8	HAMMING, JACCARD, MHJACCARD	HAMMING

Source: milvus.io docs v2.6.x — Metric Types (site/en/userGuide/search-query-get/metric.md)

5 Measured performance

5.1 VectorDBBench leaderboard (Cohere Medium 1M × 768-d, top-k = 10)

The numbers below are the official VectorDBBench results for the "$1,000 monthly cost" tier on the Cohere Medium dataset (1 million 768-dimensional dense vectors), as published on the Zilliz VectorDBBench leaderboard page.

System	QPS	P99 latency (ms)	Recall
ZillizCloud-8cu-perf	13,316.23	2.0	93.83%
Milvus-16c64g-sq4u-fp16-force_merge	9,575.69	2.3	91.89%
Milvus-16c64g-sq8-force_merge	5,973.00	2.4	91.92%
OpenSearch-16c128g-force_merge	3,055.01	7.2	90.66%
ElasticCloud-8c60g-force_merge	2,353.89	17.1	91.43%
QdrantCloud-16c64g	1,242.43	6.4	94.74%
Pinecone-p2.x8-1node	1,146.53	13.7	92.62%
OpenSearch-16c128g	950.63	13.2	91.40%
TurboPuffer	798.33	56.7	89.93%

Source: https://zilliz.com/vdbbench-leaderboard?dataset=vectorSearch (retrieved 2026-10-10)

5.2 Milvus 2.2 scale-up / scale-out benchmark

Milvus 2.2.0 官方基准报告（SIFT1M 数据集，HNSW 索引，M=8，efConstruction=200，nq=1，topk=1，ef=64）给出如下实测数据：

CPU cores	Concurrent Number	QPS	RT(TP99) / ms	RT(TP50) / ms	fail/s
8	500	7,153	127	83	0
12	300	10,248	63	24	0
16	600	14,135	85	42	0
32	600	20,281	63	28	0

Scale-up：单个 QueryNode 的 CPU 核数从 8 扩展到 32。

Replicas	Concurrent Number	QPS	RT(TP99) / ms	RT(TP50) / ms	fail/s
1	500	7,153	127	83	0
2	500	15,903	105	27	0
4	800	19,281	109	40	0
8	1,200	30,655	93	38	0

Scale-out：QueryNode 副本数从 1 扩展到 8（replica_number 等于加载集合时的副本数）。Source: milvus.io docs v2.6.x — Milvus 2.2 Benchmark Test Report (site/en/reference/benchmark.md)

6 References

[1] Milvus docs v2.6.x, “Rerankers Overview”, https://milvus.io/docs/rerankers-overview.md

[2] Milvus docs v2.6.x, “RRF Ranker”, https://milvus.io/docs/rrf-ranker.md

[3] Milvus docs v2.6.x, “Metric Types”, https://milvus.io/docs/metric.md

[4] Milvus docs v2.6.x, “Milvus 2.2 Benchmark Test Report”, https://milvus.io/docs/benchmark.md

[5] Zilliz, “VectorDBBench Leaderboard”, https://zilliz.com/vdbbench-leaderboard?dataset=vectorSearch

[6] Wang et al., “Milvus: A Purpose-Built Vector Data Management System”, SIGMOD 2021, doi:10.1145/3448016.3457550

[7] Malkov & Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs”, arXiv:1603.09320